

Travailler en parallèle sur plusieurs branches sans changer de répertoire

Principe

Un `worktree` git crée un second (ou troisième) répertoire de travail à partir du même dépôt, chacun checké sur une branche distincte. Pas de `git stash`, pas de changement de branche : deux terminaux, deux contextes, zéro friction entre eux.

Disponible depuis Git 2.5.0 (2015), nativement compatible avec Claude Code.

Commandes essentielles

```
# Créer un worktree sur une branche existante ou nouvelle
git worktree add ../monprojet-feature feature/auth
git worktree add ../monprojet-hotfix -b hotfix/login-bug

# Lister tous les worktrees actifs
git worktree list

# Nettoyer après merge
git worktree remove ../monprojet-hotfix
git worktree prune # Supprimer les références obsolètes
```

Pattern Claude Code en parallèle

```
# Terminal 1 : feature principale
cd ../monprojet-feature
claude
> "Implémente l'authentification JWT"

# Terminal 2 : hotfix urgent (simultané)
cd ../monprojet-hotfix
claude
> "Corrige le bug de timeout sur Safari"
```

Chaque instance de Claude indexe son propre `worktree` et maintient un contexte indépendant. Les fichiers sont séparés, mais ports, bases de données et services externes peuvent rester partagés.

Contexte Claude Code dans les worktrees

Fichier	Portée
<code>~/.claude/CLAUDE.md</code>	Global, partagé par tous les worktrees
<code>CLAUDE.md</code> (racine repo)	Projet, commité, partagé
<code>CLAUDE.local.md</code> (ajouter à <code>.gitignore</code>)	Instructions personnelles de ce worktree

Un `worktree` peut avoir sa propre configuration `.claude/` pour des conventions spécifiques à la branche.

Gestion des dépendances

La principale limitation : `node_modules` ou `vendor/` ne sont pas partagés automatiquement. Chaque `worktree` a besoin de ses dépendances, ce qui peut prendre de l'espace disque et du temps.

Approche recommandée : installer les dépendances séparément dans chaque `worktree`, en utilisant le cache du gestionnaire de paquets. Un lien partagé vers `node_modules` couple les branches et devient dangereux si les lockfiles ou l'état d'installation diffèrent.

```
# Depuis le nouveau worktree, avec le
gestionnaire de paquets du dépôt
pnpm install --frozen-lockfile
```

Examiner tout helper personnalisé `/git-worktree` avant d'utiliser ses options de partage des dépendances.

Quand utiliser les worktrees

Adapté pour : plusieurs features en parallèle, comparaison d'approches, revue de code pendant le développement, builds CI longs pendant qu'on continue à coder.

Pas adapté pour : changements de branches simples et rapides, équipes non familières avec l'outil (ajoute de la complexité opérationnelle), espace disque limité.

Alias shell pour navigation rapide

```
# Dans ~/.zshrc
alias wa="cd ../monprojet-feature-a"
alias wb="cd ../monprojet-feature-b"
alias wm="cd ../monprojet" # Main worktree
```

Pattern utilisé par l'équipe Claude Code en interne pour naviguer instantanément entre `worktrees` actifs.

worktree.baseRef (v2.1.133)

Contrôle le commit de base utilisé lors de la création d'un `worktree` via `--worktree`, `EnterWorktree` ou l'isolation d'agents.

Valeur	Comportement
<code>fresh</code> (défaut)	Branche depuis <code>origin/<branche-par-défaut></code> (base distante propre)
<code>head</code>	Branche depuis le HEAD local (inclut les commits non poussés)

Changement cassant v2.1.133 : `fresh` est maintenant la valeur par défaut. Si vous avez des commits non poussés à inclure dans le `worktree`, ajoutez ce bloc dans

`settings.json` :

```
{
  "worktree": {
    "baseRef": "head"
  }
}
```

Nettoyage après merge

Après merge de la branche, supprimer le `worktree` pour libérer l'espace. `git worktree prune` nettoie les références aux `worktrees` supprimés manuellement (dossier effacé sans passer par `git worktree remove`).